Compiler Design — CST302

Module 3: Intermediate-Level Topics

Operator Precedence Parsing · LR Parsing Core Concepts · LR(0) · SLR(1) · ACTION & GOTO Tables · Canonical Collection

1. Operator Precedence Parsing

Operator precedence parsing is a **type of bottom-up parsing** used for grammars where **operators have defined precedence relations**. It works by comparing precedence between terminals to decide whether to **shift or reduce**.

✔ It can handle **ambiguous grammars**. ✔ Works only on **Operator Grammars**.

2. Precedence Relations

Three relations are defined between pairs of terminals:

Relation	Notation	Meaning	Role in Parsing
Less Than	a <. b	a yields precedence to b	Marks start of handle → Shift
Equal	a =. b	Same precedence (e.g., parentheses)	Inside handle
Greater Than	a .> b	a takes precedence over b	Marks end of handle → Reduce

Key insight: **<.** marks the LEFT end of a handle, **.>** marks the RIGHT end, and **=.** appears inside the handle.

3. Operator Precedence Relation Table

For the grammar **E → E+E | E*E | (E) | id**, the relation table is:

	id	+	*	$
id	—	.>	.>	.>
+	<.	.>	<.	.>
*****	<.	.>	.>	.>
$	<.	<.	<.	—

Interpretation: ***** has higher precedence than **+** (because * .> + means * takes precedence). **$** has the lowest precedence.

4. Inserting Precedence Relations into Input String

Input: **id + id * id**

After inserting relations:

```
$ <. id .> + <. id .> * <. id .> $
```

Pair	Relation	Reason
$ and id	<.	$ has lower precedence than id
id and +	.>	id has higher precedence than +
+ and id	<.	+ yields to id
id and *	.>	id has higher precedence than *
* and id	<.	* yields to id (id is operand)
id and $	.>	id has higher precedence than $

5. Operator Precedence Parsing Algorithm

Input: String w and Precedence Relation Table **Initial Stack:** $ **Input Buffer:** w$

Steps:

Step	Condition	Action
1	$ on top, ip points to $	Return (Accept)
2	top-terminal <. b or top-terminal =. b	Shift b onto stack; advance ip
3	top-terminal .> b	Reduce: pop until <. is found
4	No relation exists	Error

Pseudocode:

```
set ip to first symbol of w$
repeat forever:
   let a = topmost terminal on stack
   let b = symbol pointed to by ip
   if a <. b or a =. b: push b; advance ip
   elif a .> b: pop stack until top-terminal <. last-popped
   elif $ on top and ip = $: return (Accept)
   else: error()
```

6. LR Parsing — Core Concepts

LR parsing is the **most powerful bottom-up parsing technique**. It can handle a large class of context-free grammars and detects errors as early as possible.

LR(k) — What does it mean?

Letter	Stands For	Meaning
L	Left-to-right	Input scanned left to right
R	Rightmost derivation	Constructs rightmost derivation in reverse
k	Lookahead	Number of input symbols looked ahead (usually 0 or 1)

7. Components of an LR Parser

Component	Description	Example
Input Buffer	Holds the entire input string	id + id * id $
Stack	Stores states and grammar symbols alternately	s0 X1 s1 X2 s2
ACTION Table	Decides: shift j / reduce / accept / error	ACTION[0, id] = s3
GOTO Table	After reduce, maps (state, non-terminal) → state	GOTO[0, A] = 2
LR Driver Program	Core control logic that runs the parsing loop	Reads ACTION/GOTO

8. ACTION and GOTO Tables

ACTION Table — ACTION[state, terminal]

Entry Value	Meaning	What Parser Does
shift j	Shift input symbol, go to state j	Push symbol and state j onto stack
reduce A → β	Reduce by production A → β	Pop 2|β| symbols; push A and GOTO result
accept	Parsing complete	Stop and announce success
error	Syntax error found	Call error recovery routine

GOTO Table — GOTO[state, non-terminal] → next state

Used **after a reduce action**. After popping the handle from the stack, the parser looks up GOTO[exposed_state, LHS_nonterminal] to find the next state.

9. LR Parsing Algorithm

Input: String w, LR Parsing Table (ACTION + GOTO)
Output: Bottom-up parse for w OR error indication
Initial state: s0 on stack, w$ in input buffer

Pseudocode:

```
push s0 onto stack
set ip to first symbol of w$

repeat forever:
   s = state on top of stack
   a = symbol pointed to by ip

   if ACTION[s, a] = shift s':
     push a then s' onto stack
     advance ip to next symbol

   elif ACTION[s, a] = reduce A → B:
     pop 2*|B| symbols from stack
     s' = new state on top
     push A then GOTO[s', A]

   elif ACTION[s, a] = accept: stop
   else: error()
```

10. Types of LR Parsers

Parser	Lookahead	Grammar Class	Table Size	Practical Use
LR(0)	None	Weakest	Small	Rarely used
SLR(1)	FOLLOW sets	Small class	Small	Academic/teaching
CLR(1)	LR(1) items	Largest class	Very Large	Theoretical
LALR(1)	**Merged LR(1)**	**Intermediate**	**Medium (=SLR)**	**Used in compilers (yacc)**

11. LR(0) Parsing

LR(0) does **not use any lookahead**. It relies purely on the current state to decide actions. It is the base for building SLR, CLR, and LALR parsers.

Steps to Construct LR(0) Parsing Table:

Step	Action
1	Write the Augmented Grammar (add S' → S)
2	Compute LR(0) Items (add dots to all productions)
3	Compute Closure of each item set
4	Compute GOTO transitions between item sets
5	Build Canonical Collection of LR(0) item sets (DFA states)
6	Fill ACTION and GOTO tables from the DFA

12. Augmented Grammar

A new start symbol **S'** is added with production **S'** → **S**. This ensures there is a unique accept state.

Example:

Production No.	Original Grammar	Augmented Grammar
0 (new)	—	S' → S
1	S → AA	S → AA
2	A → aA	A → aA
3	A → b	A → b

13. LR(0) Items

An **LR(0) item** is a production with a **dot (·)** somewhere in the RHS. The dot shows **how much of the production has been parsed**.

Item	Meaning
A → · a A	Nothing parsed yet; expecting 'a'
A → a · A	'a' has been parsed; expecting non-terminal A
A → a A ·	Entire RHS parsed → this is a REDUCE item

14. Closure Operation

If an item set I contains an item of the form **A** → α **·** **B** β, then for every production **B** → γ, add **B** → **·** γ to I.

Example:

```
Start with: S' → · S
S is a non-terminal, so add productions of S with dot at start:
S → · A A
A is a non-terminal, so add productions of A:
A → · a A
A → · b
```

15. GOTO Operation

GOTO(I, X) moves the dot across symbol X in all items of set I and then takes the closure of the result.

Example:

```
If I contains: A → a · A b
Then GOTO(I, A) gives:
A → a A · b (dot moved across A)
Then closure is computed on this new set.
```

16. Canonical Collection of LR(0) Items

The canonical collection is the **set of all LR(0) item sets** (states). Each item set is a **state in the DFA**. GOTO transitions form the edges of this DFA.

For grammar S → AA, A → aA | b:

State	Items in Set	GOTO Transitions
I0	S'→·S, S→·AA, A→·aA, A→·b	GOTO(I0,S)=I1, GOTO(I0,A)=I2, GOTO(I0,a)=I3, GOTO(I0,b)=I4
I1	S'→S·	Accept state
I2	S→A·A, A→·aA, A→·b	GOTO(I2,A)=I5, GOTO(I2,a)=I3, GOTO(I2,b)=I4
I3	A→a·A, A→·aA, A→·b	GOTO(I3,A)=I6, GOTO(I3,a)=I3, GOTO(I3,b)=I4
I4	A→b·	Reduce by A→b
I5	S→AA·	Reduce by S→AA
I6	A→aA·	Reduce by A→aA

17. LR(0) Parsing Table Rules

Rule	Condition	Table Entry
Shift Rule	Item: A → α · a β in state i, GOTO(Ii, a) = Ij	ACTION[i, a] = shift j
Reduce Rule	Item: A → α · in state i (dot at end)	ACTION[i, a] = reduce A→α (all terminals a)
Accept Rule	Item: S' → S · in state i	ACTION[i, $] = accept
GOTO Rule	GOTO(Ii, A) = Ij (A is non-terminal)	GOTO[i, A] = j

18. SLR(1) Parsing

SLR(1) is an improvement over LR(0). The only difference is in the **Reduce rule**: instead of reducing for all terminals, SLR reduces only for terminals in **FOLLOW(A)**.

Aspect	LR(0)	SLR(1)
Reduce condition	Reduce for ALL terminals	Reduce only for terminals in FOLLOW(A)
Conflicts	More conflicts possible	Fewer conflicts than LR(0)
Extra step needed	None	Compute FOLLOW sets
Grammar class	Smallest	Slightly larger than LR(0)

Steps to Construct SLR(1) Table:

Step	Action
1	Write the Augmented Grammar
2	Compute LR(0) items (same as LR(0))
3	Compute FOLLOW sets for all non-terminals
4	Build ACTION table: reduce only in FOLLOW(A) columns
5	Build GOTO table (same as LR(0))

19. SLR(1) Example

Grammar: S → AA, A → aA | b

FOLLOW sets:

Non-Terminal	FOLLOW Set	Reason
S	{ $ }	S is start symbol; followed by end marker
A	{ a, b, $ }	A appears before another A in S→AA (so FIRST(A)={a,b}), and at end

SLR(1) Parsing Table:

State	a (ACTION)	b (ACTION)	$ (ACTION)	A (GOTO)	S (GOTO)
0	S3	S4	—	2	1
1	—	—	Accept	—	—
2	S3	S4	—	5	—
3	S3	S4	—	6	—
4	R3	R3	R3	—	—
5	—	—	R1	—	—
6	R2	R2	R2	—	—

R1 = reduce by S→AA, R2 = reduce by A→aA, R3 = reduce by A→b, S3/S4 = shift to state 3/4

20. Limitation of SLR Parser

SLR uses FOLLOW sets as lookahead, which can be **too general**. A terminal may be in FOLLOW(A) but not actually follow A in the specific parsing context, leading to **false reductions**.

Conflict Type	Cause	SLR Solution	Still Fails When
Shift-Reduce	Both shift and reduce valid for same terminal	Use FOLLOW(A) for reduce	FOLLOW is too broad
Reduce-Reduce	Two different reduces valid	Use FOLLOW(A)	Both A's have same FOLLOW

Solution: Use CLR(1) which carries exact lookahead inside each LR item, or LALR(1) which merges CLR states to reduce table size.

Complete Concept Map — Module 3 Intermediate

Topic	Key Concept	Output/Result	Exam Weight
Operator Precedence	3 relations: <. =. .>	Handle identification via relation table	Medium
LR Parsing	L-R-k definition, 4 components	Parser structure understanding	High
ACTION/GOTO Tables	shift/reduce/accept/error entries	Parsing decisions at each step	Very High
LR(0) Items	Dot notation in productions	DFA states for parser	High
Closure & GOTO ops	Expand dot across non-terminals/symbols	Canonical collection states	High
SLR(1)	Reduce only in FOLLOW columns	Smaller conflicts than LR(0)	Very High
SLR Limitation	FOLLOW too broad → conflicts	Motivates CLR and LALR	Medium